Practical – 4

AIM : To develop a game-playing agent for Tic-Tac-Toe using the Minimax algorithm with Alpha-Beta pruning.

Lab theory : Minimax is a decision-making algorithm used in two-player, turn-based games. It assumes that both players play optimally.

In Tic-Tac-Toe:

- The AI player tries to maximize its score.
- The human player tries to minimize the AI's score.
- The algorithm examines possible future moves and selects the best move.

The scores can be assigned as:

- AI wins → +10
- Human wins → -10
- Draw → 0

Alpha-Beta Pruning

Alpha-Beta pruning is an optimization of the Minimax algorithm. It avoids checking branches of the game tree that cannot affect the final decision.

- Alpha (α): Best value that the maximizing player can guarantee.
- Beta (β): Best value that the minimizing player can guarantee.
- If $\alpha \geq \beta$, the remaining branches are skipped.

This makes Minimax faster while producing the same result.

Code :

```
board = [" "] * 9


def win(p):
    for a,b,c in [(0,1,2),(3,4,5),(6,7,8),
                  (0,3,6),(1,4,7),(2,5,8),
                  (0,4,8),(2,4,6)]:
        if board[a] == board[b] == board[c] == p:
            return True
    return False


def minimax(alpha, beta, max_player):
    if win("O"): return 1
    if win("X"): return -1
    if " " not in board: return 0


    if max_player:
        best = -999
        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                best = max(best, minimax(alpha, beta, False))
```

```
  board[i] = " "
                alpha = max(alpha, best)
                if alpha >= beta:
                    break
        return best
    else:
        best = 999
        for i in range(9):
            if board[i] == " ":
                board[i] = "X"
                best = min(best, minimax(alpha, beta, True))
                board[i] = " "
                beta = min(beta, best)
                if alpha >= beta:
                    break
        return best


def best_move():
    best, move = -999, 0
    for i in range(9):
        if board[i] == " ":
            board[i] = "O"
```

```
 score = minimax(-999, 999, False)
            board[i] = " "
            if score > best:
                best, move = score, i
    return move


print("Tic-Tac-Toe: You = X, AI = O")


for turn in range(9):
    print("\n", board[0:3])
    print(board[3:6])
    print(board[6:9])


    if turn % 2 == 0:
        pos = int(input("Enter position (1-9): ")) - 1
        if board[pos] == " ":
            board[pos] = "X"
        else:
            print("Invalid move")
            continue
```

```
else:
        m = best_move()
        board[m] = "O"
        print("AI position:", m + 1)

    if win("X"):
        print("You Win!")
        break
    if win("O"):
        print("AI Wins!")
        break
else:
    print("Draw!")
```

output :

```
    Tic-Tac-Toe: You = X, AI = O

    [' ', ' ', ' ']
    [' ', ' ', ' ']
    [' ', ' ', ' ']
```

Enter position (1-9): 1

AI position: 5

['X', ' ', ' ']

[' ', 'O', ' ']

[' ', ' ', ' ']

Enter position (1-9): 2

AI position: 3

['X', 'X', 'O']

[' ', 'O', ' ']

[' ', ' ', ' ']

Conclusion :

The Tic-Tac-Toe game-playing agent was successfully implemented using Minimax and Alpha-Beta pruning. The AI selects the best possible move while pruning unnecessary game-tree branches